

Assignment 7

Secure Network Application Development Using TCP

Name: Tanvi Chaudhari

Roll No: 5130108

ISEA Phase 3 – Tezpur University Summer Internship

GitHub Repository:

<https://github.com/tanvi-24c/ISEA-Phase3-TezpurUniversity-Assignment6>

1. Objective

The objective of this assignment is to enhance the GUI-based multi-client TCP chat application built in Assignment 6 by adding practical application-security mechanisms. This includes username/password authentication, secure password storage using SHA-256 hashing, prevention of duplicate logins, input validation, protection against repeated failed login attempts, and session management with secure event logging. The focus is on application-level security rather than advanced cryptography, and the underlying networking code from Assignment 6 has been reused and extended rather than rewritten.

2. Security Features Implemented

Authentication (Task 1)

Every client must send a LOGIN command with a username and password before it can send chat messages or any other command. Unauthenticated clients are rejected with an "ERROR|Not authenticated" response for any command other than LOGIN.

Secure Password Storage (Task 2)

Passwords are never stored or transmitted in plain text on disk. `generate_users.py` hashes each account password with `hashlib.sha256()` before writing it to `users.csv`, and the server compares the SHA-256 hash of the submitted password against the stored hash rather than comparing raw passwords.

Duplicate Login Prevention (Task 3)

The server keeps an in-memory dictionary, `active_users`, mapping each logged-in username to its socket connection. If a second LOGIN attempt arrives for a username that is already present in `active_users`, the server rejects it with "ERROR|User already logged in elsewhere" and logs a `DUPLICATE_LOGIN` event.

Input Validation (Task 4)

Usernames are validated against a regular expression (3–20 alphanumeric/underscore characters). Passwords must be non-empty and under 128 characters. Chat messages are capped at 1024 bytes, and any command outside the `SUPPORTED_COMMANDS` whitelist (LOGIN, MSG, LOGOUT, PING) is rejected with an `INVALID_INPUT` log entry.

Failed Login Protection (Task 5)

The server tracks `failed_attempts` per username. After 5 consecutive invalid-credential attempts, the account is temporarily blocked for 60 seconds (`BLOCK_DURATION_SECONDS`). Further LOGIN attempts during the block window return the remaining lockout time and are logged as `LOGIN_BLOCKED`.

Session Management & Secure Logging (Task 6)

Clients can end a session explicitly with LOGOUT. The server also enforces a 120-second inactivity timeout (`INACTIVITY_TIMEOUT_SECONDS`), after which an idle authenticated session is force-logged-out. Every security-relevant event (connect, login success/failure, duplicate login, blocked

login, invalid input, message, logout, disconnect, session timeout) is appended to security_log.txt with a timestamp, event type, username and IP – passwords and password hashes are never written to this log.

3. System Architecture

The application keeps the same two-component architecture as Assignment 6: a multi-threaded TCP server (server.py) and a Tkinter GUI client (client_gui.py). Networking logic remains independent of the GUI so any client implementation could reuse the same line-based protocol over TCP port 5000.

- server.py – accepts TCP connections, spawns a daemon thread per client (handle_client), and holds shared state (active_users, failed_attempts, clients) protected by a threading.Lock (state_lock).
- client_gui.py – Tkinter interface with a login screen and a chat screen; sends pipe-delimited commands (LOGIN|user|pass, MSG|text, LOGOUT, PING) and reads newline-terminated responses from the server.
- generate_users.py – one-time helper script that creates users.csv containing only SHA-256 password hashes.
- users.csv – the user credential store (username, hash).
- security_log.txt – append-only audit log of all security-relevant events.

Network testing was carried out on a Mininet single-switch, five-host topology (sudo mn --topo single,5), with connectivity verified using nodes, net and pingall before running the server and client on separate hosts.

4. Implementation

Key implementation details, taken directly from server.py:

Password hashing:

```
def hash_password(password: str) -> str:
    return hashlib.sha256(password.encode("utf-8")).hexdigest()
```

Duplicate login check:

```
with state_lock:
    already_logged_in = username in active_users
if already_logged_in:
    log_event("DUPLICATE_LOGIN", username, addr, "rejected")
    return False, "ERROR|User already logged in elsewhere"
```

Failed-attempt logout:

```
record["count"] += 1
if record["count"] >= MAX_FAILED_ATTEMPTS:
    record["blocked_until"] = time.time() + BLOCK_DURATION_SECONDS
```

Configuration constants used throughout: MAX_MESSAGE_SIZE = 1024 bytes, USERNAME_REGEX = `^[A-Za-z0-9_]{3,20}$`, MAX_FAILED_ATTEMPTS = 5, BLOCK_DURATION_SECONDS = 60, INACTIVITY_TIMEOUT_SECONDS = 120.

5. Testing

Both scripted protocol tests and manual GUI testing were performed to verify every task. The screenshots below show the client login screen, a successful authenticated login, the duplicate/failed-login and account-lockout behaviour, input validation rejecting a bad request, an authenticated chat message being relayed, and a clean logout.

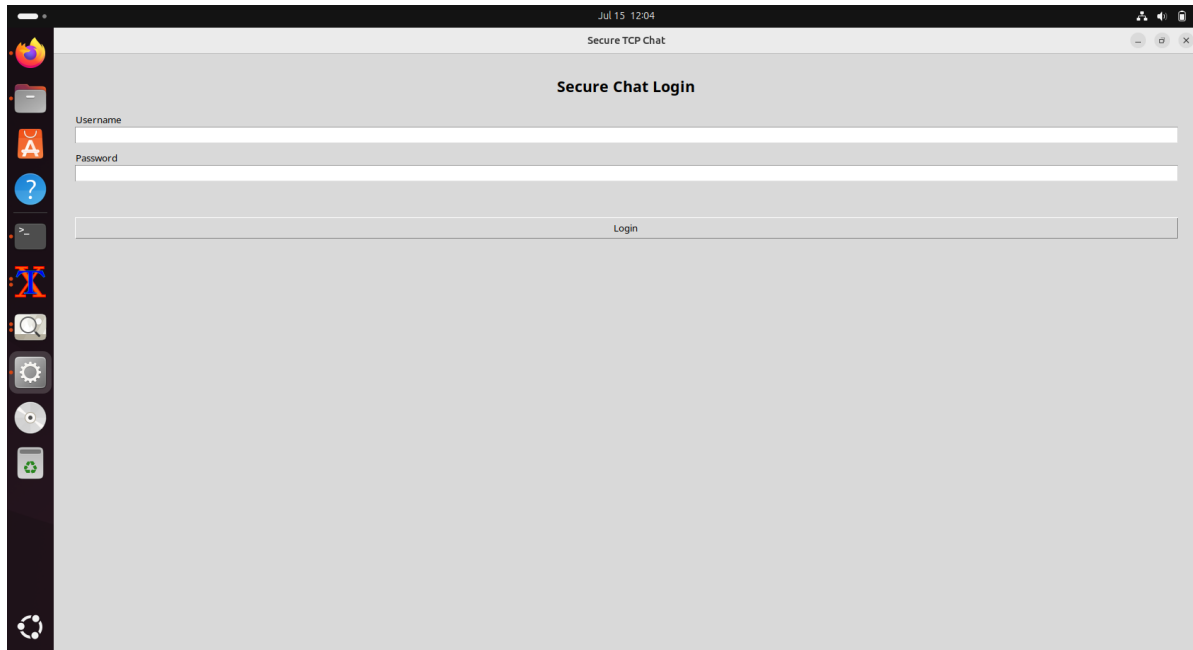


Fig 1: Client login screen (username/password fields)

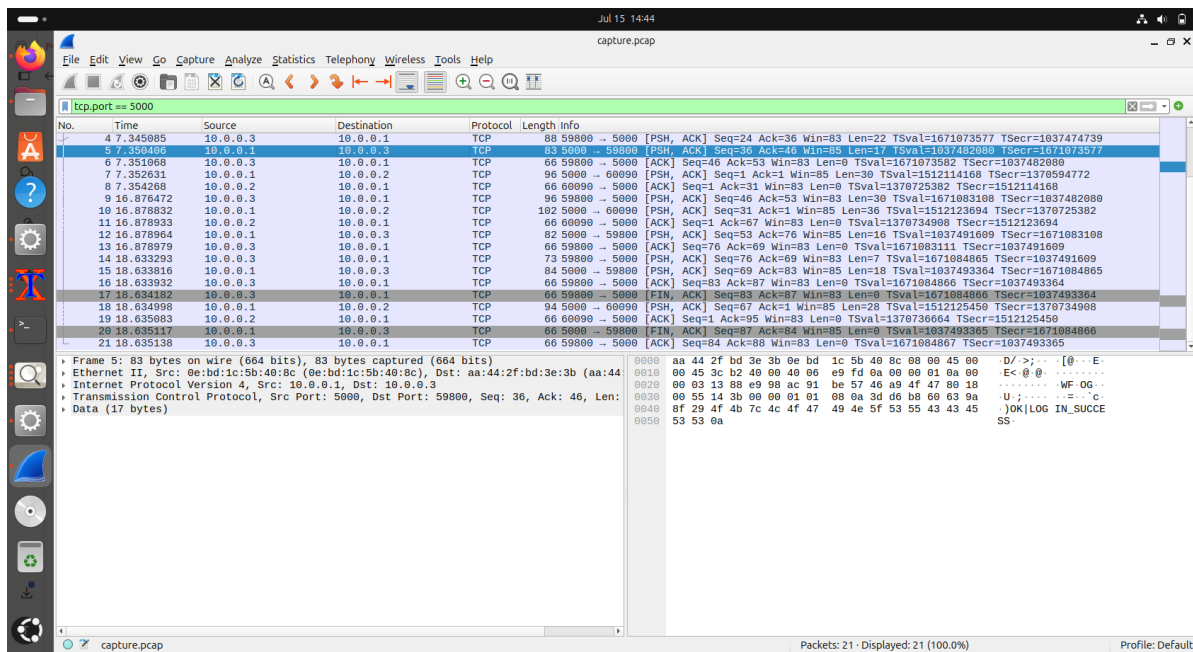


Fig 2: Successful authentication and chat window

A screenshot of a web application window titled "Secure TCP Chat". The window contains a "Secure Chat Login" form. The form has two input fields: "Username" with the value "tanvi" and "Password" with masked characters "*****". Below the password field, a red error message reads "User already logged in elsewhere". At the bottom of the form is a "Login" button.

Fig 3: Failed login attempt handling

A screenshot of the same "Secure TCP Chat" window. The "Username" field still contains "tanvi" and the "Password" field is masked with "*****". The red error message now reads "Account temporarily locked. Try again in 54s". The "Login" button remains at the bottom.

Fig 4: Account temporarily locked after 5 failed attempts

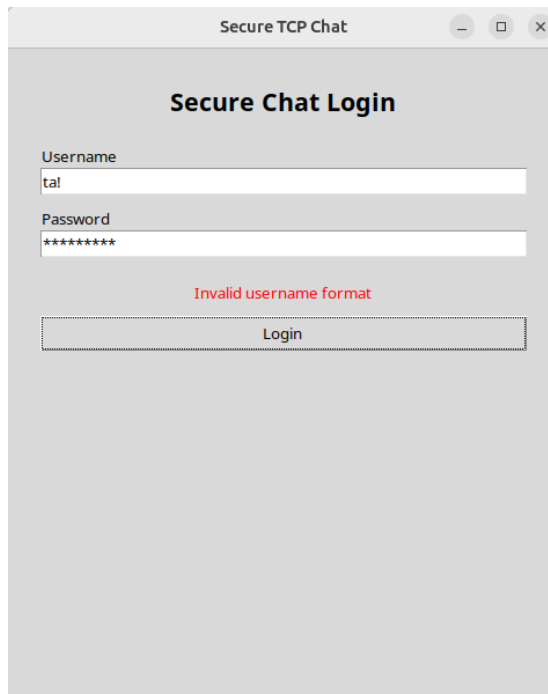


Fig 5: Invalid input rejected by the server

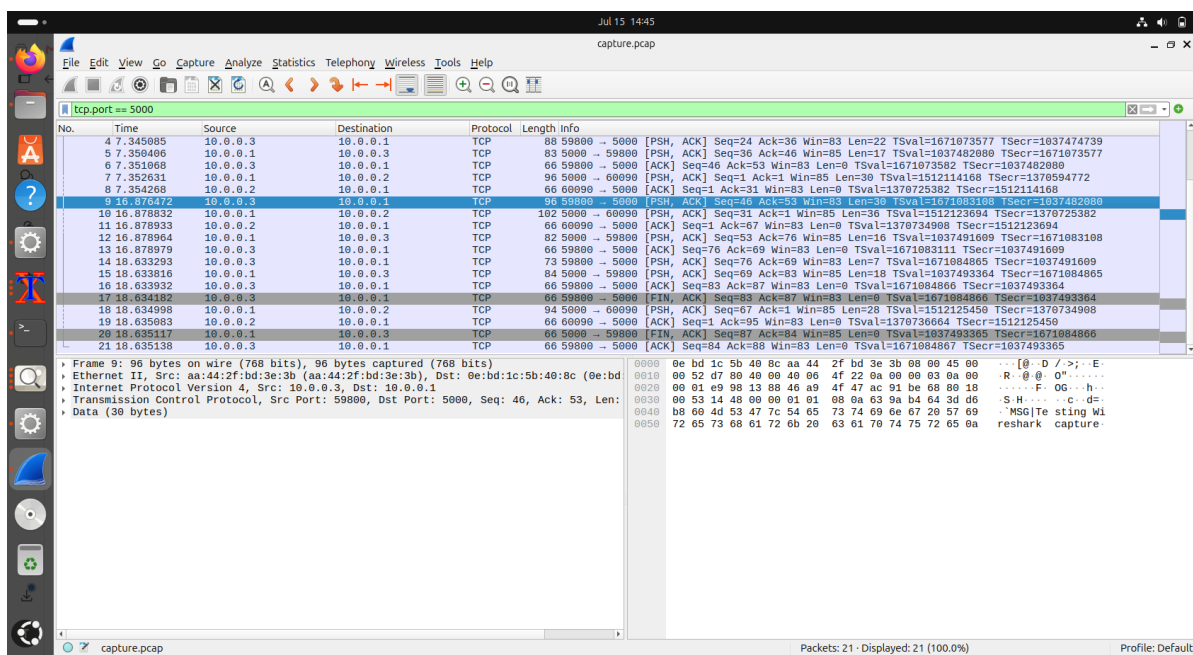


Fig 6: Authenticated chat message relayed between clients

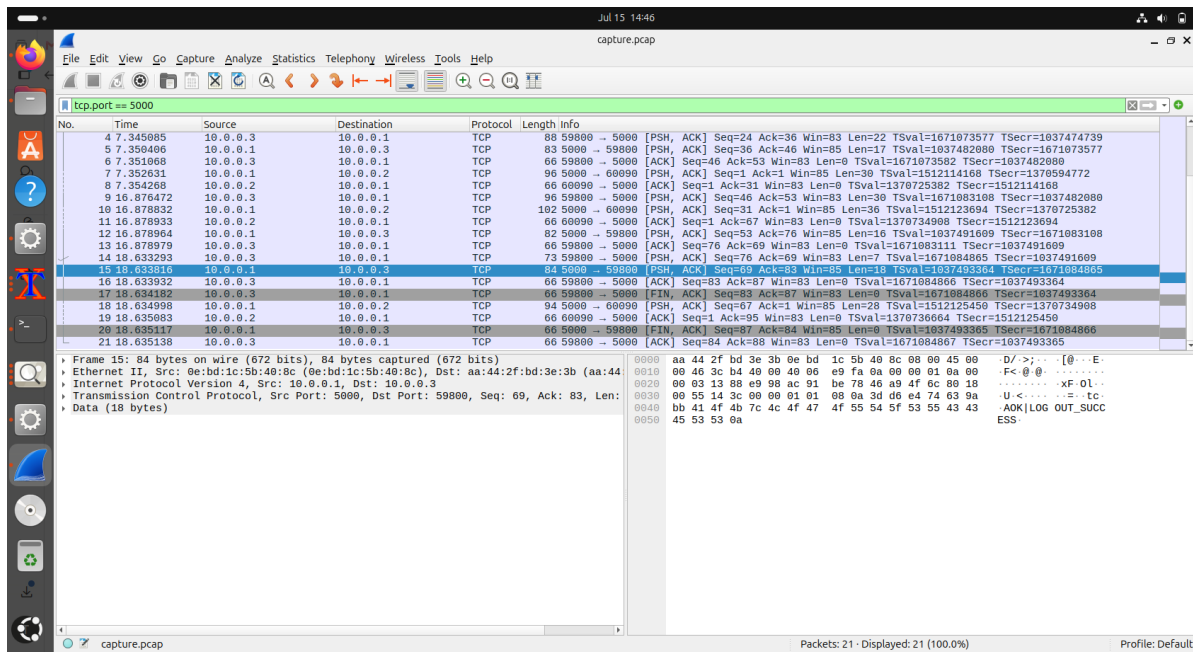


Fig 7: Client logout

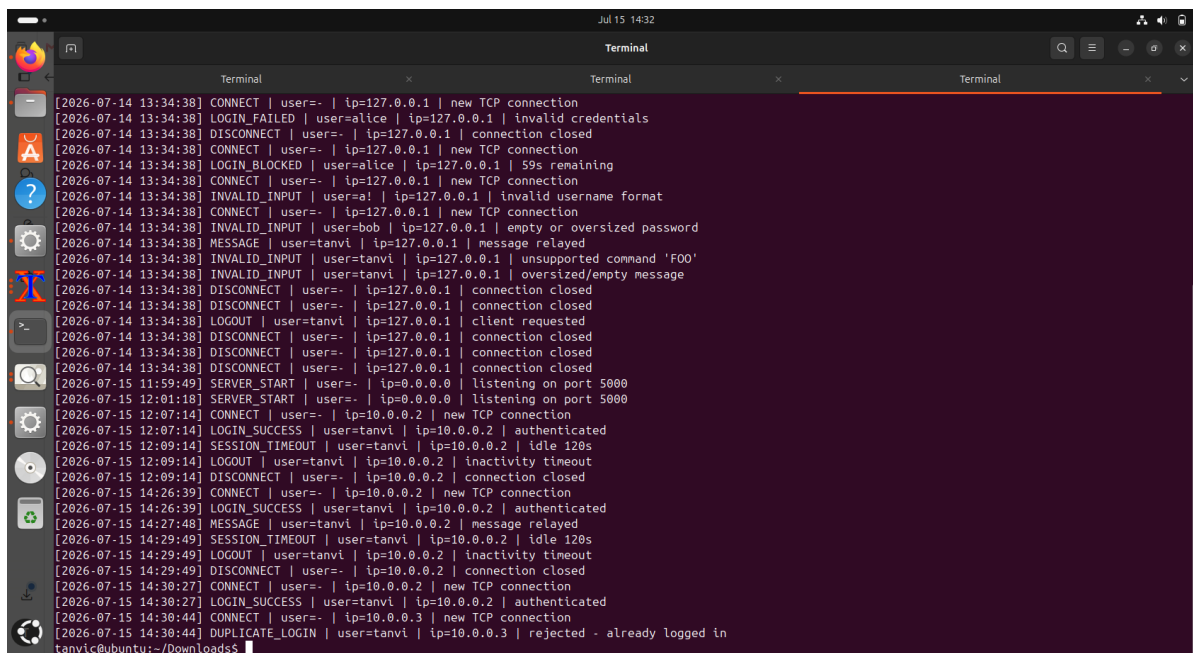


Fig 8: security_log.txt showing recorded events

A sample excerpt from security_log.txt confirms that events are logged without ever recording plaintext or hashed passwords:

```
[2026-07-14 13:34:38] LOGIN_SUCCESS | user=tanvi | ip=127.0.0.1 | authenticated
[2026-07-14 13:34:38] DUPLICATE_LOGIN | user=tanvi | ip=127.0.0.1 | rejected - already logged in
[2026-07-14 13:34:38] LOGIN_FAILED | user=alice | ip=127.0.0.1 | invalid credentials
[2026-07-14 13:34:38] LOGIN_BLOCKED | user=alice | ip=127.0.0.1 | 59s remaining
[2026-07-15 12:09:14] SESSION_TIMEOUT | user=tanvi | ip=10.0.0.2 | idle 120s
```


6. Wireshark Verification

Packet captures were taken on the loopback/Mininet interface using the display filter `tcp.port == 5000`, covering the login exchange, an authenticated chat message, and the logout sequence. Because the transport is plain TCP (no TLS), the capture also visibly confirms at the protocol design level why the application must never place plaintext passwords on the wire in a way that is logged or displayed – only the SHA-256 hash comparison happens server-side, and the capture was used to double-check no password material leaks into server console output or logs.

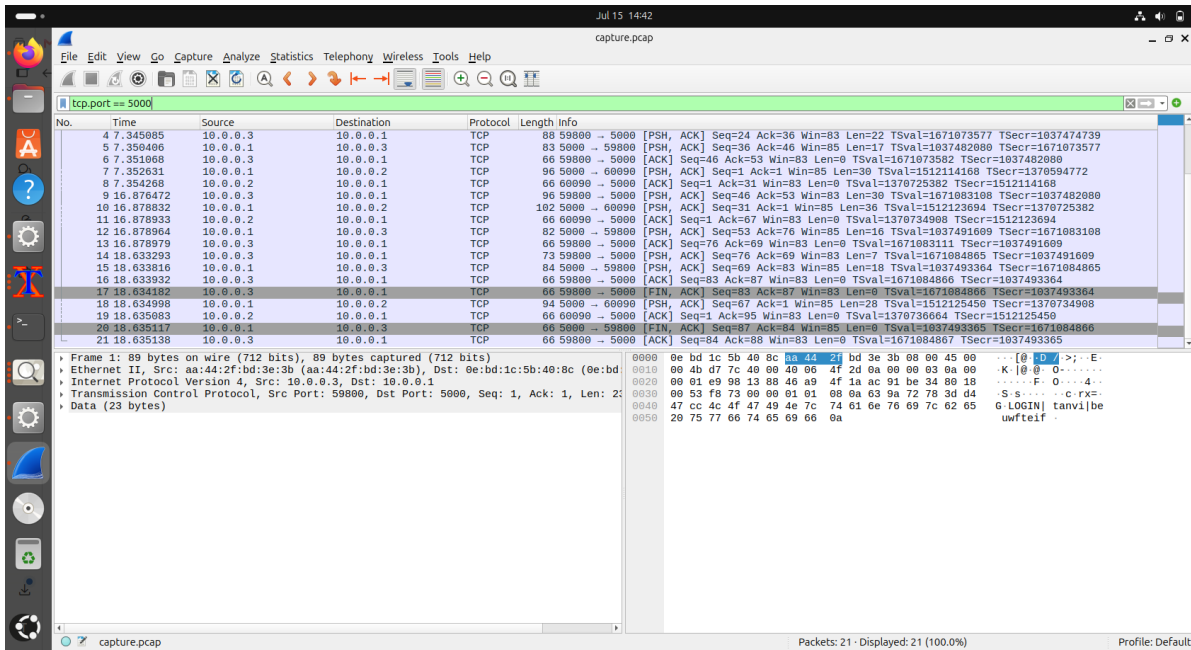


Fig 9: Wireshark capture filtered on `tcp.port == 5000`

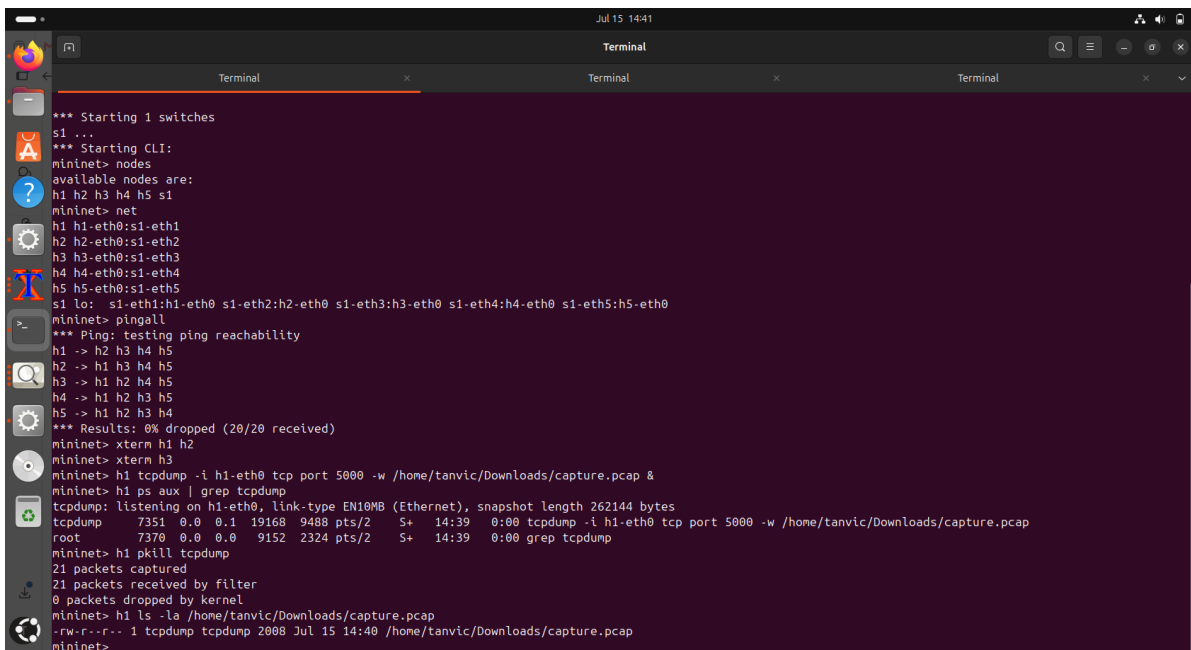


Fig 10: Server console output during the captured session

7. Conclusion

This assignment extended the Assignment 6 multi-client TCP chat application with a practical, layered set of application-security controls: authentication gated on SHA-256-hashed credentials, duplicate-login prevention, strict input validation, temporary lockout after repeated failed logins, and session/inactivity management backed by a password-free audit log. All required tasks (1–7) were implemented directly on top of the existing networking core without rewriting it, and the behaviour was verified through both scripted tests and Wireshark packet captures on `tcp.port == 5000`. The project demonstrates that meaningful security improvements can be layered onto a simple TCP application without introducing heavy cryptographic machinery, while still following key principles such as never storing or logging plaintext passwords.

GitHub Repository: <https://github.com/tanvi-24c/ISEA-Phase3-TezpurUniversity-Assignment6>